

The same convention is used for method names:

```
def _internalMethod(self):  
  
def _dontCallMeFromClientSoftware(self):
```

Again, this is only a convention; there is no enforcement. If any client software makes a call to a method with a name beginning with an underscore, Python will allow it, but there is a good chance that doing so will lead to unexpected errors.

More Explicitly Private

Python does allow for a more explicit level of privatization. To disallow client software from directly accessing your data, you create an instance variable name that starts with two underscores.

Suppose we create a class named `PrivatePerson` with an instance variable `self.__privateData` that should never be accessed from outside an object:

```
# PrivatePerson class  
  
class PrivatePerson():  
  
    def __init__(self, name, privateData):  
        self.name = name  
        ❶ self.__privateData = privateData  
  
    def getName(self):  
        return self.name  
  
    def setName(self, name):  
        self.name = name
```

We can then create a `PrivatePerson` object, passing in some data that we wish to keep private ❶. Attempting to access the `__privateData` instance variable directly from client software, like this:

```
usersPrivateData = oPrivatePerson.__privateData
```

will generate an error: